

QueryGenie: Making LLM-Based Database Querying Transparent and Controllable

Longfei Chen
School of Information Science and
Technology
ShanghaiTech University
Shanghai, China
chenlf@shanghaitech.edu.cn

Shenghan Gao
School of Information Science and
Technology
ShanghaiTech University
Shanghai, China
gaoshh1@shanghaitech.edu.cn

Shiwei Wang
Shenzhen Yishi Huolala Technology
Co., Ltd
shenzhen, China
three.wang@huolala.cn

Ken Lin
Shenzhen Yishi Huolala Technology
Co., Ltd
Shenzhen, China
adam.lin@huolala.cn

Yun Wang
Microsoft Research
Hong Kong, China
wangyun@microsoft.com

Quan Li^{*}
School of Information Science and
Technology
ShanghaiTech University
Shanghai, Shanghai, China
liquan@shanghaitech.edu.cn

Abstract

Conversational user interfaces powered by large language models (LLMs) have significantly lowered the technical barriers to database querying. However, existing tools still encounter several challenges, such as misinterpretation of user intent, generation of hallucinated content, and the absence of effective mechanisms for human feedback—all of which undermine their reliability and practical utility. To address these issues and promote a more transparent and controllable querying experience, we proposed *QueryGenie*, an interactive system that enables users to monitor, understand, and guide the LLM-driven query generation process. Through incremental reasoning, real-time validation, and responsive interaction mechanisms, users can iteratively refine query logic and ensure alignment with their intent.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; **Natural language interfaces**.

Keywords

User interface, large language model, data query

ACM Reference Format:

Longfei Chen, Shenghan Gao, Shiwei Wang, Ken Lin, Yun Wang, and Quan Li. 2025. QueryGenie: Making LLM-Based Database Querying Transparent and Controllable. In *The 38th Annual ACM Symposium on User Interface Software and Technology (UIST Adjunct '25)*, September 28–October 01, 2025, Busan, Republic of Korea. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3746058.3758982>

^{*}Quan Li is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

UIST Adjunct '25, Busan, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2036-9/25/09

<https://doi.org/10.1145/3746058.3758982>

1 Introduction

Conversational user interfaces powered by large language models (LLMs), such as *Vanna* [9] and *DataLine* [2], have significantly lowered the technical barriers to data querying, allowing non-expert users to access and explore databases with ease [4, 6]. By removing the need to write complex query languages or operate specialized tools, these systems enable users to interact with data more flexibly through natural language.

Despite their great potential, LLM-powered data querying tools still face significant reliability challenges, such as generating hallucinated content [10], misinterpreting user intent [3, 7], or misunderstanding database schemas [6]. These issues often lead to inaccurate query results and, in turn, undermine user trust. Most existing research [1, 8, 11] focuses on addressing these problems by improving model performance—for example, through prompt engineering or fine-tuning to optimize output quality.

However, solely relying on model improvements is not sufficient to address users' confusion and uncertainty during real-world interactions. We argue that enabling users to see how the LLM reasons—understanding why and how a query is generated—is key to enhancing system transparency and building trust.

To this end, we developed *QueryGenie*, an interactive system designed to help users understand and intervene in the query generation process of LLMs. The system follows a three-stage framework: **Intention Confirmation** – **Query Generation** – **Query Validation**. First, the system visualizes the mapping between the user's natural language input and the database schema, revealing how the LLM interprets user intent. Users can inspect and adjust this mapping to ensure that their intent is accurately captured before the system proceeds to generate structured query language (SQL). The query generation process adopts a chain-of-thought (CoT) approach, breaking down the query into a sequence of simpler, interpretable steps. Each step is accompanied by an explanation and an executable SQL statement, allowing users to examine and validate the logic in real time. If issues are detected, users can directly edit the SQL or provide feedback to the LLM for revision.

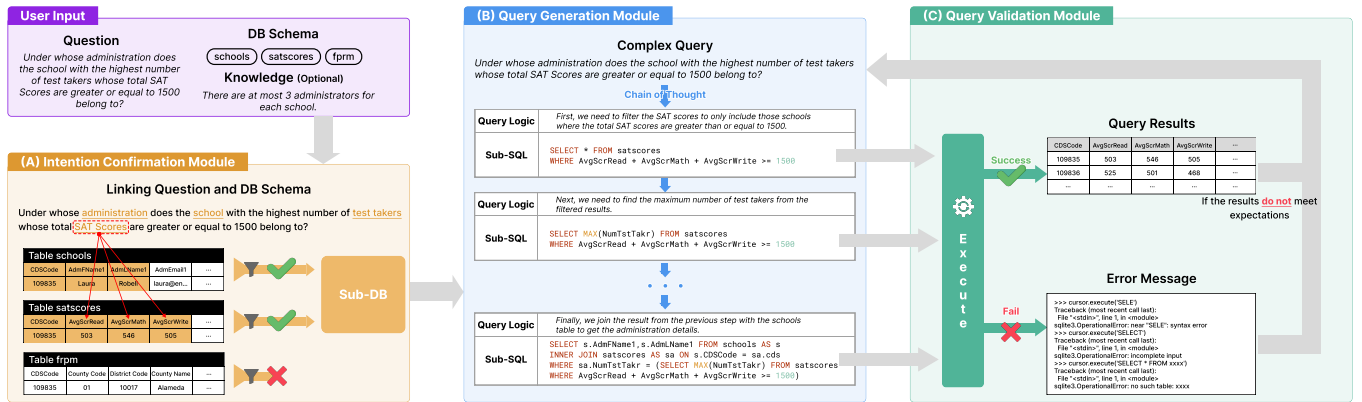


Figure 1: The framework of *QueryGenie* consists of three key modules: (A) Intention Confirmation Module, which interprets user intentions from natural language input and maps them to the database schema; (B) Query Generation Module, which constructs query logic and converts it into precise SQL statements; and (C) Query Validation Module, which executes the SQL statements and returns the results.

2 System Framework

We propose a novel human-AI collaboration framework (Figure 1) aimed at enabling more controllable and transparent data querying.

User Input. To facilitate the subsequent discussion, we define the user input as a triplet $X = (Q, S, \mathcal{K})$, where Q denotes the query question, S denotes the database schema, and $\mathcal{K}$ denotes external knowledge.

Intention Confirmation Module. This module verifies whether the LLM can accurately understand the user’s intended query expressed in natural language. As shown in Figure 1 (A), the system first guides the LLM to map entities in the query Q (e.g., “SAT Scores”) with corresponding fields in the database schema S (e.g., AvgScrRead, AvgScrMath and AvgScrWrite)—a process known as schema linking [5]. Users then review and adjust these mappings to ensure semantic alignment. If any mappings are incorrect, users can manually correct them or modify the input to trigger a new round of linking. Once accurate mappings between Q and S are established, the system filters out irrelevant portions of the schema, retaining only a focused subset S' for subsequent steps. This approach not only reduces the likelihood of irrelevant information affecting the query generation but also shortens the prompt length, thereby improving reasoning efficiency.

Query Generation Module. This module enhances the LLM’s reasoning ability by using the CoT method to break down complex queries into a series of intermediate steps. As shown in Figure 1 (B), the module guides the LLM to decompose complex queries into simpler sub-queries, with each sub-query corresponding to a specific sub-SQL statement. This approach not only helps users better understand the query logic but also improves the transparency of the generation.

Query Validation Module. This module works in conjunction with the Query Generation Module to individually check and verify each sub-query in the reasoning chain. As shown in Figure 1 (C), the module executes SQL statements within the database and provides real-time results. This allows users to immediately observe the effects of each sub-query and quickly identify and correct any potential errors. If the result of a SQL query at a particular step

does not meet expectations, users can provide immediate feedback by modifying the query logic or directly editing the SQL statements. The system will then update the subsequent steps in the reasoning chain accordingly. This incremental validation approach effectively mitigates the issue of error propagation in complex queries. By reviewing and adjusting each sub-query generation step, users can swiftly identify and rectify logical errors, preventing issues from becoming difficult to trace in the final SQL output.

3 QueryGenie

Based on the proposed framework, we developed a demo system called *QueryGenie*, which integrates a database environment with an enhanced conversational interface to help users collaborate more effectively with LLMs in completing data querying tasks.

3.1 Database Panel

In the system, the database panel allows users to quickly connect to local databases and intuitively explore their structure. As shown in Figure 2, users can import a database by selecting its path, and the system will automatically display all tables and fields with keyword-based filtering support. The panel also integrates a UML diagram that visualizes the relationships between tables, making it easier to understand the database schema. In addition, users can select specific tables of interest, and the system will extract their schema information for subsequent interactions with the LLM. This targeted approach prevents irrelevant information from interfering with the LLM’s reasoning, improving overall efficiency—especially when working with large databases.

3.2 Enhanced Conversational Interface

The system leverages an enhanced conversational interface to support transparent and controllable collaboration between user and LLM for data querying. When a user submits an initial query (Figure 2 A), the system provides both the query input and the schema of the selected tables to the LLM, prompting it to establish necessary mappings between natural language entities and database fields. These mappings are then visualized for the user, with matched

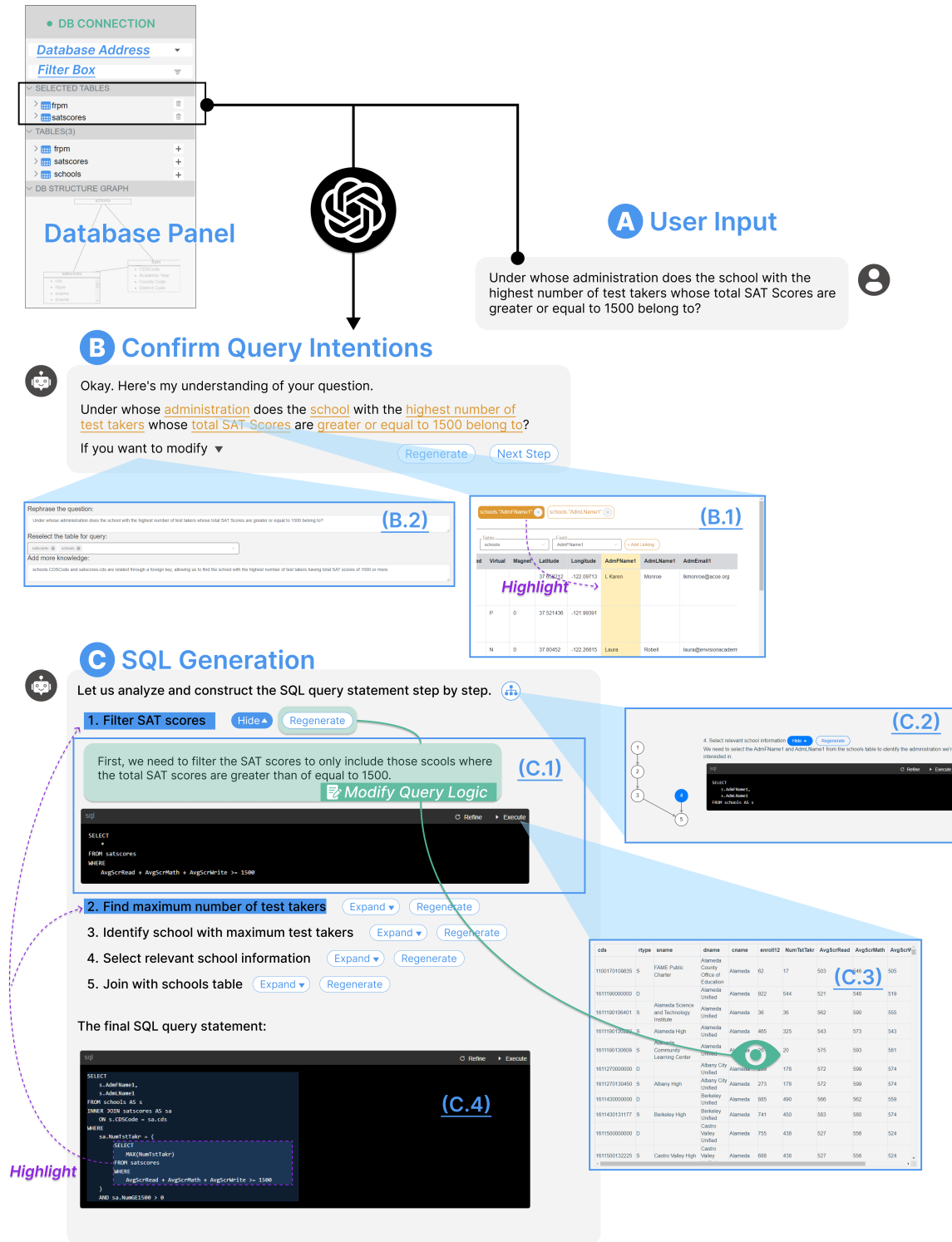


Figure 2: The user interface of QueryGenie.

entities highlighted using yellow underlines (Figure 2 B). Users can click on these highlighted entities to view the corresponding field

names and sample data. If any mappings are incorrect, users can

either modify the associated fields (Figure 2 B.1) or adjust the query input (Figure 2 B.2) to trigger a new round of schema linking.

Once the user confirms that the LLM has accurately understood their intent, the system guides the LLM to generate the query logic through a CoT approach. As shown in Figure 2 (C), the original query is decomposed into five sub-queries, which are displayed in sequence. Users can click the **Expand** (Expand) button to view detailed explanations and corresponding sub-SQL statements for each step (Figure 2 C.1). Notably, due to dependencies between reasoning steps—e.g., in Figure 2 (C), Step 2 builds upon the result of Step 1—the query forms a hierarchical tree structure rather than a simple linear sequence. To reflect this structure, we introduce a tree-based visualization (Figure 2 C.2) that illustrates the hierarchical relationships and dependency paths among sub-queries. Users can toggle between the textual and tree views via the **Tree** button, enabling a more intuitive understanding of how each step contributes to the overall query logic.

Since the system is connected to a database, all SQL statements generated in the conversational interface can be executed directly to support real-time validation. As shown in Figure 2 (C.3), users can click the **Execute** (Execute) button to view the results of any SQL query. Two common interaction scenarios are supported: (1) If the SQL has syntax errors, users can click the **Refine** (Refine) button to prompt the LLM to self-correct and return a revised statement; (2) If the SQL is executable but the results do not align with the intended logic, users can adjust the logic of the relevant step and click the **Regenerate** (Regenerate) button, prompting the LLM to generate an updated SQL based on the modified logic. Given the dependency across sub-queries, editing one step may affect subsequent ones. Although this “cascade effect” is logically consistent, users typically do not expect earlier steps to be affected. Therefore, when a node l_i in the logic chain $l_1 \rightarrow l_2 \rightarrow \dots \rightarrow l_n$ is modified, the system preserves all prior nodes $l_1 \rightarrow \dots \rightarrow l_{i-1}$, while regenerating only the subsequent ones $l_{i+1} \rightarrow \dots \rightarrow l_n$.

At the bottom of the interface (Figure 2 C.4), the system displays the final SQL statement. To make its structure more understandable, a color-coded visualization is applied: darker colors indicate deeper levels of nesting, and each colored block corresponds to a specific reasoning step. Users can hover over these blocks to view the associated logic, supporting better comprehension and traceability of the final SQL construction process.

4 Preliminary Evaluation

We conducted a within-subjects user study to preliminarily evaluate the effectiveness of *QueryGenie*. A total of 12 participants (**P1–12**, 9 male, 3 female; age: 22.19 ± 3.32) were asked to complete 6 data querying tasks using both *QueryGenie* and *Vanna* [9], respectively. All tasks were randomly sampled from the BIRD benchmark [6]. To ensure fairness, both systems used the same underlying LLM, *GPT-4o*. Additionally, the order in which participants used the two systems was counterbalanced to minimize learning effects. During the process, we recorded the completion time and accuracy for each task. After finishing all tasks, participants were invited to take part in a semi-structured interview to provide additional feedback.

Task Performance. Participants using *QueryGenie* achieved an overall accuracy of 90.3% (65/72), with an average completion time

of 203 seconds per task. In contrast, *Vanna* yielded an accuracy of 66.7% (48/72) and an average task time of 130 seconds. These results suggest that while *QueryGenie* requires more time to complete each task, it enables users to produce significantly more accurate queries. We observed that participants often spent more time using *QueryGenie* to review and understand intermediate reasoning steps generated by the system. Although this process increased task duration, most participants viewed it as a worthwhile trade-off—sacrificing a small amount of time for improved reliability and greater transparency in the querying process.

Usability Feedback. In post-study interviews, participants consistently expressed a preference for *QueryGenie*. Many noted that the system “*integrated multiple useful features*” (7/12), “*enhanced their understanding of how the LLM generates data queries*” (7/12), and “*made them feel more confident when completing tasks*” (8/12). The majority of participants (11/12) found *QueryGenie* easy to learn and smooth to use. As **P3** remarked, “*The system [QueryGenie] introduces some additional features, but I can clearly understand their purpose—they’re designed to better support task completion.*” During the study, we also observed that all participants were able to quickly adapt to the system and became proficient after completing just one or two tasks.

5 Conclusion

In this study, we proposed an innovative interaction framework and demo system designed to enhance human–LLM collaboration in data querying tasks. Built upon traditional conversational LLMs, our approach integrated visualizations and real-time interactive feedback, enabling users to better understand the model’s reasoning process and intervene at critical points. This enhances both the accuracy and transparency of query generation.

References

- [1] Shuaichen Chang and Eric Fosler-Lussier. 2023. How to prompt llms for text-to-sql: A study in zero-shot, single-domain, and cross-domain settings. *arXiv preprint arXiv:2305.11853* (2023).
- [2] DataLine.app. 2024. <https://dataline.app/>.
- [3] Yujian Gan, Xinyun Chen, Qiuping Huang, Matthew Purver, John R Woodward, Jinxia Xie, and Pengsheng Huang. 2021. Towards robustness of text-to-SQL models against synonym substitution. *arXiv preprint arXiv:2106.01065* (2021).
- [4] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2024. Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2024).
- [5] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. 2020. Re-examining the Role of Schema Linking in Text-to-SQL. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6943–6954.
- [6] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [7] Alisa Liu, Zhaofeng Wu, Julian Michael, Alane Suhr, Peter West, Alexander Koller, Swabha Swayamdipta, Noah A Smith, and Yejin Choi. 2023. We’re afraid language models aren’t modeling ambiguity. *arXiv preprint arXiv:2304.14399* (2023).
- [8] Mohammadreza Pourreza and Davood Rafiei. 2024. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems* 36 (2024).
- [9] Vanna.AI. 2024. <https://vanna.ai/>.
- [10] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817* (2024).
- [11] Alex Zhuang, Ge Zhang, Tianyu Zheng, Xinrun Du, Junjie Wang, Weiming Ren, Stephen W Huang, Jie Fu, Xiang Yue, and Wenhui Chen. 2024. StructLM: Towards Building Generalist Models for Structured Knowledge Grounding. *arXiv preprint arXiv:2402.16671* (2024).